



ACRON Methodology Part-4

فاز 3: Customer Domain



در Part-3 ، فاز 3 تا قدم 31 پیش رفت

32- فایل `apps/customers/tests/test_api.py` را باز کنید و در بالای فایل،
ماژول `datetime` را اضافه کنید:

```
from rest_framework.test import APITestCase
from rest_framework import status
import datetime # این خط اضافه شود

from apps.accounts.models import CustomUser
```

33- این دو متد جدید را به انتهای کلاس `CustomerMeApiTest` اضافه کنید:

```
:def test_patch_invalid_phone_number(self)
    # ۱. احراز هویت کاربر
    self.client.force_authenticate(user=
self.user)
```

```
    # ۲. ارسال درخواست ویرایش با شماره ت
لفن غلط (کمتر از 10 کاراکتر)
    response = self.client.patch
    , '/api/customers/me/'
    }
    'phone_number': '123'
    , {
    'format='json
    (
```

```
    # ۳. بررسی اینکه آیا سیستم خطای 400
(Bad Request) داده است؟
    self.assertEqual
    , response.status_code
    status.HTTP_400_BAD_REQUEST
    (
```

```
    # ۴. بررسی اینکه آیا خطا دقیقاً مربوط
به فیلد phone_number است؟
    self.assertIn('phone_number', respon
se.data)
```

```
:def test_patch_future_birth_date(self)
    self.client.force_authenticate(user=
```

```

self.user)

# ۱. ساخت یک تاریخ در آینده (مثلاً 10
روز بعد از امروز)
future_date = (datetime.date.today()
+ datetime.timedelta(days=10)).strftime('%Y-
%m-%d')

# ۲. ارسال تاریخ آینده به سرور
)response = self.client.patch
, '/api/customers/me/'
}
'birth_date': future_date'
,{
'format='json
(

# ۳. بررسی اینکه آیا سیستم خطای 400
داده است؟
)self.assertEqual
,response.status_code
status.HTTP_400_BAD_REQUEST
(

# ۴. بررسی اینکه آیا خطا مربوط به فیلد
birth_date است؟
self.assertIn('birth_date', respons
e.data)

```

نتیجه شبیه به این خواهد بود:

```
$ python manage.py test apps/customers/tests/
Found 12 test(s).
Creating test database for alias 'default'...
System check identified no issues (0 silenced).

.....
-----
-----
Ran 12 tests in 63.191s

OK
Destroying test database for alias 'default'...
```

کالبدشکافی سریع کد (چرا اینطور نوشتیم؟):

- `status.HTTP_400_BAD_REQUEST`: وقتی کلاینت (فرانت‌اند یا موبایل) در معماری REST، دیتای نامعتبری می‌فرستد، سرور باید کد 400 را برگرداند. متد `.is_valid()` در DRF این کار را به صورت خودکار انجام می‌دهد.
- `self.assertIn('phone_number', response.data)`: ما فقط نمی‌خواهیم بدانیم که سیستم خطا داده است، بلکه می‌خواهیم مطمئن شویم سیستم دقیقاً فهمیده است که مشکل از `phone_number` است. در این حالت شبیه این است DRF کد/م فیلد است. خروجی خطای `phone_number` چک می‌کند که کلید `assertIn` دستور `["Phone number is too short."]` داخل پاسخ سرور وجود داشته باشد.

فاز 4: Product Domain

نحوه آپلود، تغییر (جابه جایی)، حذف تصویر توسط ادمین برای محصولات سایت
 نحوه آپلود، تغییر (جابه جایی)، حذف تصویر توسط کاربران برای پروفایل، محصولات
 شخصی، ارسال تصویر در چت شخصی بین کاربران.

<https://pillow.readthedocs.io/en/stable/handbook/tutorial.html#identify-image-files>

<https://www.geeksforgeeks.org/python/imagefield-django-models/>

مدل‌ها:

- Category
- Brand
- Product

- ProductImage
- Comment
-
- UUID
- Slug
- Inventory
- Optimization
- select_related
- prefetch_related
- N+1 Problem

مدلهایی که باید طراحی کنیم:

- **Category** (دسته‌بندی‌ها)
- **Brand** (برندها)
- **Product** (محصولات)
- **ProductImage** (گالری تصاویر)

مفاهیم کلیدی که با آن‌ها درگیر می‌شویم:

- **Slug**: (SEO Friendly) های زیبا URL برای ایجاد
- **تصاویر**: آپلود و مدیریت فایل‌های مدیا (Pillow).
- **Inventory** (موجودی): مدیریت موجودی کالا.
- **Optimization** (بهینه‌سازی دیتابیس): ارسال ده‌ها کوئری N+1 جلوگیری از مشکل (اضافی به دیتابیس).



تصمیمات معماری برای **Category** (دسته‌بندی) و **Brand** (برند)

۱. مفهوم Slug (اسلاگ) چیست و چرا واجب است؟

در فازهای قبلی (مثل Customer) ما با ID یا UUID کار می‌کردیم. مثلاً `/api/customers/1`. اما برای محصولات و دسته‌بندی‌ها، URL باید برای موتورهای جستجو (SEO) و انسان‌ها خوانا باشد.

به جای: `/acron.com/category/12`

ما این را می‌خواهیم: `/acron.com/category/smart-phones` می‌گویند `Slug`. ما در تمام مدل‌های این فاز به فیلد `Slug` نیاز داریم. به کلمه `smart-phones`

۲. دسته‌بندی‌های تو در تو (Nested Categories)

در یک فروشگاه واقعی، ما "کالای دیجیتال" داریم، زیرمجموعه آن "موبایل" است و زیرمجموعه آن "گوشی اپل".
برای پیاده‌سازی این ساختار درختی، مدل `Category` باید بتواند به خودش اشاره کند (Self-Referential ForeignKey).

۳. نیاز به تصاویر

چون دسته‌بندی‌ها، برندها و محصولات نیاز به عکس دارند، جنگو برای فیلد تصویر (`ImageField`) به یک کتابخانه پردازش تصویر در پایتون به نام `Pillow` نیاز دارد.



شروع کدنویسی فاز ۴

35- نصب کتابخانه مدیریت تصاویر (Pillow):

در ترمینال (در حالی که محیط مجازی فعال است) دستور زیر را وارد کنید:

```
pipenv install pillow
```

چرا این کار را کردیم؟

جنگو بدون این کتابخانه اجازه استفاده از `models.ImageField` را نمی‌دهد و خطا پرتاب می‌کند، زیرا برای اعتبارسنجی اینکه آیا فایل آپلود شده واقعاً یک عکس است یا یک فایل مخرب، به موتور پردازشی `Pillow` متکی است.

36- فعال‌سازی اپلیکیشن در `base.py`:

فایل `config/settings/base.py` را باز کنید و اپلیکیشن `products` را از کامنت خارج کنید:

```
] = INSTALLED_APPS
... #
, 'apps.accounts'
, 'apps.customers'
```

```
'apps.products' , # این خط از کامنت خارج  
شود  
[
```

37- ساخت مدل‌های `Category` و `Brand` :

فایل `apps/products/models.py`

را باز کنید و کدهای زیر را بادقت بنویسید (خط به خط را در ادامه کالبدشکافی می‌کنیم):

```
from django.db import models

class Category(models.Model):
    # دسته‌بندی والد (برای ساختار درختی)
    parent = models.ForeignKey(
        'self',
        on_delete=models.PROTECT,
        null=True,
        blank=True,
        related_name='children'
    )

    name = models.CharField(max_length=255)

    # اسلاگ برای URL های سئو-محور
    slug = models.SlugField(max_length=255,
        unique=True, allow_unicode=True)

    description = models.TextField(blank=True,
        e)
```

```
# تصویر دسته بندی
image = models.ImageField(upload_to='categories/%Y/%m/', blank=True, null=True)

class Meta
    verbose_name_plural = 'Categories'

def __str__(self)
    return self.name

class Brand(models.Model)
    name = models.CharField(max_length=255)
    slug = models.SlugField(max_length=255,
        unique=True, allow_unicode=True)
    image = models.ImageField(upload_to='brands/%Y/%m/', blank=True, null=True)

def __str__(self)
    return self.name
```



کالبدشکافی دقیق کدها (بدون Vibe Coding)

- `parent = models.ForeignKey('self', ...)`:
به جنگو می‌گویید که این دسته‌بندی به یک دسته‌بندی دیگر در همین جدول `'self'` کلمه یعنی اگر دسته‌بندی والد نداشت (مثل خود "کالای" `null=True, blank=True` متصل است. دیجیتال" که بالاترین سطح است)، می‌تواند خالی بماند.
- `on_delete=models.PROTECT` در `parent`:
بود، ادمین با حذف کردن دسته‌بندی "کالای CASCADE نگذاشتیم؟ اگر CASCADE چرا دیجیتال"، اشتباهاً تمام زیرمجموعه‌های آن (موبایل، لپ‌تاپ و...) را هم پودر می‌کرد!

باعث می‌شود جنگو بگوید: "تا زمانی که این دسته، زیرمجموعه دارد، اجازه `PROTECT` حذف آن را ندارید."

- `related_name='children'` : این یعنی اگر من یک دسته والد باشم (مثلاً موبایل)، با دستور `category.children.all()` می‌توانم تمام زیرمجموعه‌هایم (سامسونگ، اپل، شیائومی) را استخراج کنم.
- `allow_unicode=True` در `SlugField`: فقط حروف `SlugField`، بسیار مهم برای سایت‌های ایرانی! به صورت پیش‌فرض انگلیسی را قبول می‌کند. وقتی این پارامتر را می‌دهیم، می‌توانیم اسلاگ فارسی داشته باشیم (مثلاً `acron.com/category/گوشی-موبایل`).
- `upload_to='categories/%Y/%m/'` : وقتی عکسی آپلود می‌شود، جنگو همه را در یک پوشه نمی‌ریزد. این پارامتر به جنگو این (مثلاً `categories/2026/07/`) پوشه‌بندی کند (`%m`) و ماه (`%Y`) می‌گوید بر اساس سال کار باعث می‌شود در آینده که هزاران عکس داشتید، سرور لینوکس شما در پیدا کردن فایل‌ها گُند نشود.
- `verbose_name_plural = 'Categories'` : به انتهای نام مدل‌ها در پنل ادمین اضافه می‌کند. اگر `s` جنگو به صورت پیش‌فرض یک که از نظر املاي انگلیسی غلط `Categorys` این خط را ننویسیم، در پنل ادمین می‌نویسد است. با این خط آن را اصلاح کردیم.

حالا که کدهای مدل `Category` و `Brand` را نوشتیم، دیتابیس ما هنوز از وجود این جداول بی‌خبر است.

قدم بعدی ما دو بخش دارد:

ابتدا این تغییرات را به دیتابیس MySQL تزریق می‌کنیم و سپس آن‌ها را به پنل ادمین جنگو متصل می‌کنیم تا بتوانیم اولین دسته‌بندی‌ها و برندها را بسازیم.



قدم اول: انتقال مدل‌ها به دیتابیس (Migration Lifecycle)

زمانی که ما یک مدل پایتونی می‌نویسیم، دیتابیس (MySQL) مستقیماً آن را نمی‌فهمد. ما باید از سیستم اسمارت Migration در جنگو استفاده کنیم تا این کدهای پایتون را به دستورات SQL (مثل `CREATE TABLE`) تبدیل کند.

38- اجرای کامندهای ساخت و اعمال ترنزکشن‌ها در ترمینال:

ابتدا مطمئن شوید در محیط مجازی (`pipenv shell`) هستید، سپس دستور زیر را بزنید:

```
python manage.py makemigrations products
```

نتیجه شبیه زیر در ترمینال خواهد بود

```
$ python manage.py makemigrations products
Migrations for 'products':
  apps\products\migrations\0001_initial.py
    + Create model Brand
    + Create model Category
```



پشت صحنه چه رخ داد؟

جنگو مدل‌های اپلیکیشن `products` را بررسی کرد و یک فایل جدید به نام `initial.py_0001` در پوشه `/apps/products/migrations` ساخت.

این فایل صرفاً یک «نقشه ساخت» یا Blueprint به زبان پایتون است و هنوز تاثیری روی دیتابیس نگذاشته است.

39- حالا برای اعمال واقعی این نقشه روی MySQL، دستور زیر را اجرا کنید:

```
python manage.py migrate
```

نتیجه شبیه زیر خواهد بود:

```
$ python manage.py migrate
Operations to perform:
  Apply all migrations: accounts, admin, auth, contenttypes, customers, products, sessions
Running migrations:
  Applying products.0001_initial... OK
```

پشت صحنه چه رخ داد؟ جنگو نقشه پایتونی را خواند، آن را به دستورات SQL تبدیل کرد و جداول `products_category` و `products_brand` را در دیتابیس `acron` ایجاد کرد. اگر خروجی

را دیدید، یعنی همه چیز با موفقیت انجام شده است. Applying products.0001_initial... OK



این قسمت اختیاری است و مخصوص تست های خودکار گیت هاب است

40- مجدداً داخل ترمینال این دستور رو بزن و داخل گیت هاب ارسال کن

```
pipenv requirements > requirements.txt
```

نتیجه تا اینجا شبیه زیر خواهد بود: requirements.txt

```
i https://pypi.org/simple-  
'asgiref==3.11.1; python_version >= '3.9  
'django==6.0.6; python_version >= '3.12  
djangoRESTframework==3.17.1; python_version  
'>= '3.10  
djangoRESTframework-simplejwt==5.5.1; python  
'_version >= '3.9  
'mysqlclient==2.2.8; python_version >= '3.10  
'pillow==12.3.0; python_version >= '3.10  
'pyjwt==2.13.0; python_version >= '3.9  
'sqlparse==0.5.5; python_version >= '3.8  
'tzdata==2026.2; python_version >= '2
```



قدم دوم: مدیریت در پنل ادمین و جادوی خودکارسازی اسلاگ (Slug)

اگر الان به پنل ادمین جنگو (/admin/) بروید، اثری از دسته بندی ها و برندها نمی بینید. باید آن ها را رجستر (ثبت) کنیم. اما می خواهیم یک کار حرفه ای انجام دهیم؛ ادمین سایت نباید مجبور باشد خودش دستی اسلاگ بنویسد!

ما کاری می‌کنیم که وقتی ادمین نام دسته‌بندی را تایپ می‌کند، جنگو به صورت خودکار اسلاگ آن را بسازد.

41- فایل `apps/products/admin.py` را باز کنید و کدهای زیر را بنویسید:

Python

```
from django.contrib import admin
from .models import Category, Brand

admin.register(Category)@
:class CategoryAdmin(admin.ModelAdmin)
# ۱. مشخص کردن ستون‌هایی که در لیست ادمین
نمایش داده می‌شوند
list_display = ['name', 'slug', 'paren
t']

# ۲. جادوی پر شدن خودکار اسلاگ بر اساس نا
م
prepopulated_fields = {'slug': ('nam
e', )}

admin.register(Brand)@
:class BrandAdmin(admin.ModelAdmin)
list_display = ['name', 'slug']
prepopulated_fields = {'slug': ('nam
e', )}
```



کالبدشکافی دقیق کدهای ادمین (بدون Vibe Coding)

- `@admin.register(Category)` : پایتونی است. به زبان ساده به جنگو می‌گوید: «کلاس (Decorator) این یک دکوراتور در نظر بگیر و آن را در `Category` را به عنوان تنظیمات مدیریتی مدل `CategoryAdmin` ادمین ثبت کن.»
- `list_display` : را نشان می‌دهد. `__str__` به صورت پیش فرض، جنگو در لیست مدل‌ها فقط خروجی متد با این آپشن، ما یک جدول منظم می‌سازیم که ستون‌های نام، اسلاگ و والد را مجزا نمایش دهد تا پنل ادمین خوانا تر شود.
- `prepopulated_fields = {'slug': ('name',)}` : این مهم‌ترین بخش است! این تاپل به جنگو دستور می‌دهد که در صفحه ساخت دسته‌بندی پشت صحنه اجرا کند. وقتی ادمین در فیلد (JS) یا برند، یک جاوااسکریپت را با متن "کالای- `slug` می‌نویسد "کالای دیجیتال"، جنگو به صورت خودکار و زنده فیلد گذاشته بودیم، این `allow_unicode=True` دیجیتال " پر می‌کند. از آنجا که ما در مدل قبلی قابلیت به شکل بی‌نظیری با کلمات فارسی هم کار خواهد کرد.



تست و مشاهده نتیجه

حالا سرور پروژه را روشن کنید:

```
python manage.py runserver
```

به آدرس `/http://127.0.0.1:8000/admin` بروید و با سوپربوزری که در فاز ۱ ساختید وارد شوید.

وارد بخش **Categories** شوید و روی **Add Category** کلیک کنید.

تلاش کنید یک دسته‌بندی فارسی (مثلاً: لپ تاپ و تبلت) بسازید و ببینید چطور فیلد اسلاگ به صورت خودکار پر می‌شود.

یک بار هم یک دسته‌بندی والد بسازید و در دکمه بعدی، دسته‌بندی جدیدی بسازید و والد آن را روی دسته‌بندی قبلی تنظیم کنید تا ساختار درختی را به چشم ببینید.



ساخت Product Model



بررسی و تصمیمات معماری (Architecture Design)

1. **تصویر اصلی محصول:** چون هر محصول دقیقاً یک تصویر اصلی دارد، منطقی‌ترین کار این است که فیلد `main_image` را مستقیماً داخل خود مدل `Product` قرار دهیم.
2. **۱۰ تصویر/ویدیوی فرعی (گالری مالتی‌مدیا):** از آنجایی که تعداد این فایل‌ها متغیر است (بین ۰ تا ۱۰ عدد) و هر فایل می‌تواند تصویر یا ویدیو باشد، نباید ۱۰ فیلد مجزا در مدل محصول بسازیم (این یک اشتباه رایج و ضد الگو است). راهکار اصولی، ایجاد یک مدل مجزا به نام `ProductMedia` است که با یک رابطه یک‌به‌چند (`ForeignKey`) به محصول متصل می‌شود.
3. **تشخیص طول زمان ویدیو:** برای اینکه بفهمیم ویدیو چند ثانیه است، پایتون به صورت پیش‌فرض ابزاری ندارد. ما باید یک کتابخانه سبک برای پردازش ویدیو به نام `opencv-python` نصب کنیم تا بتواند متادیتای ویدیو را بخواند و مدت زمان آن را به ثانیه به ما بدهد.



قدم اول: نصب ابزار پردازش ویدیو

42- در ترمینال خود دستور زیر را وارد کنید تا کتابخانه مورد نیاز برای سنجش زمان ویدیو نصب شود:

```
pipenv install opencv-python
```



قدم دوم: نوشتن ولیدیتورها و مدل‌ها

43- فایل `apps/products/models.py` را باز کنید. کدهای زیر را جایگزین کنید تا قدم به قدم آن‌ها را کالبدشکافی کنیم:

```
import cv2
from django.db import models
from django.core.exceptions import ValidationError
nError
```

۱. ساخت ولیدیتور سفارشی برای فایل‌های فرعی (حجم و زمان ویدیو)

```

: def validate_media_file(file)
# الف) بررسی حجم فایل (400 مگابایت به بالا)
max_size_mb = 400
max_size_bytes = max_size_mb * 1024 * 1024
: if file.size > max_size_bytes
    raise ValidationError(f"حجم فایل نمیتواند بیشتر از {max_size_mb} مگابایت باشد.")

# ب) بررسی مدت زمان ویدیو (اگر فایل ویدیو بود)
file_name = file.name.lower
if file_name.endswith(('.mp4', '.mkv', '.avi', '.mov'))
# باز کردن موقت فایل ویدیو با OpenCV برای خواندن فریمها
video = cv2.VideoCapture(file.temporarily_file_path())

# به دست آوردن تعداد کل فریمها و نرخ فریم (FPS)
frames = video.get(cv2.CAP_PROP_FRAME_COUNT)
fps = video.get(cv2.CAP_PROP_FPS)

# محاسبه زمان به ثانیه (اگر fps صفر نباشد)
: if fps > 0
    duration_seconds = frames / fps

```

```

د if duration_seconds > 120: # 2
    قيقه = 120 ثانیه
    raise ValidationError(
        "مدت ز مان ویدیو نمیتواند بیشتر از ۲ دقیقه باشد.")
    video.release

```

```

class Category(models.Model)
    # کدهای قبلی دسته‌بندی بدون تغییر اینجا به
    # مانند
    parent = models.ForeignKey('self', on_de
lete=models.PROTECT, null=True, blank=True,
related_name='children')
    name = models.CharField(max_length=255)
    slug = models.SlugField(max_length=255,
unique=True, allow_unicode=True)
    description = models.TextField(blank=True
e)
    image = models.ImageField(upload_to='cat
egories/%Y/%m/', blank=True, null=True)

```

```

class Meta
    'verbose_name_plural = 'Categories

```

```

def __str__(self)
    return self.name

```

```

class Brand(models.Model)
    # کدهای قبلی برند بدون تغییر اینجا بهمانند
    د

```

```

name = models.CharField(max_length=255)
slug = models.SlugField(max_length=255,
unique=True, allow_unicode=True)
image = models.ImageField(upload_to='brands/%Y/%m/', blank=True, null=True)

```

```

: def __str__(self)
: return self.name

```

```

# ۲. ساخت مدل اصلی محصول (Product)
: class Product(models.Model)
category = models.ForeignKey(Category, on_delete=models.PROTECT, related_name='products')
brand = models.ForeignKey(Brand, on_delete=models.PROTECT, related_name='products')
name = models.CharField(max_length=255)
slug = models.SlugField(max_length=255, unique=True, allow_unicode=True)
()description = models.TextField
price = models.DecimalField(max_digits=10, decimal_places=2)
# استفاده از Decimal بر
ای قیمت‌ها الزامی است
inventory = models.PositiveIntegerField
# (default=0) موجودی کالا نباید منفی باشد

# تصویر اصلی محصول (اجباری)
main_image = models.ImageField(upload_to='products/main/%Y/%m/')

```

```

created_at = models.DateTimeField(auto_now_add=True)
updated_at = models.DateTimeField(auto_now=True)

def __str__(self)
return self.name

# ۳. ساخت مدل گالری فرعی (تصاویر و ویدیوها)
class ProductMedia(models.Model)
    MEDIA_TYPES = (
        ('image', 'Image')
        , ('video', 'Video')
    )

    product = models.ForeignKey(Product, on_delete=models.CASCADE, related_name='media_gallery')
    media_type = models.CharField(max_length=10, choices=MEDIA_TYPES)

# استفاده از FileField چون هم عکس را قبول میکند و هم ویدیو را
file = models.FileField(upload_to='products/gallery/%Y/%m/', validators=[validate_media_file])

# متد clean برای محدود کردن تعداد کل مدیاهای یک محصول به حداکثر ۱۰ عدد
def clean(self)

```

```

()super().clean
# شمارش مدیاهای فعلی این محصول در دی
تا بیس (بدون احتساب رکوردی که الان دارد ذخیره
(میشود)
:if self.product_id
existing_media_count = ProductMe
dia.objects.filter(product=self.product).exc
()lude(pk=self.pk).count
:if existing_media_count >= 10
raise ValidationError("شما ز
می‌توانید بیشتر از ۱۰ فایل فرعی (تصویر/ویدیو)
برای یک محصول آپلود کنید.")

:def save(self, *args, **kwargs)
# قبل از ذخیره نهایی در دیتابیس، حتم
اً متد clean را صدا می‌زنیم تا ولیدیشن‌ها اجرا
شوند
()self.full_clean
super().save(*args, **kwargs)

```



کالبدشکافی خط به خط کدهای جدید (بدون Vibe Coding)

- تابع `validate_media_file`: جنگو به ما اجازه می‌دهد توابعی بنویسیم که فایل ورودی را قبل از ذخیره شدن بازرسی کنند.
 - حجم فایل را به بایت برمی‌گرداند. با ضرب کردن `1024 * 1024 * 400` آن `file.size` را به بایت تبدیل کردیم تا با حجم واقعی فایل مقایسه شود.
 - وقتی فایلی در جنگو آپلود می‌شود و حجمش زیاد: `file.temporary_file_path()` ذخیره می‌کند. ما با این (`/tmp` مثل) است، جنگو آن را در پوشه موقت سیستم عامل می‌دهیم (OpenCV) `cv2` متد، آدرس فیزیکی فایل را به کتابخانه

• فرمول `frames / fps` : ویدیوها مجموعه‌ای از تصاویر پشت سر هم (فریم) هستند. اگر تعداد کل فریم‌ها را بر تعداد فریم در ثانیه (FPS) تقسیم کنیم، مدت زمان دقیق ویدیو به ثانیه به دست می‌آید. ما چک کردیم که این عدد بزرگتر از ۱۲۰ ثانیه (۲ دقیقه) نباشد.

• **فیلد `price = models.DecimalField(...)` :**

توسعه‌دهندگان تازه‌کار برای قیمت از `FloatField` استفاده می‌کنند که اشتباه است. فلوته‌ها در محاسبات ریاضی دیتابیس دچار خطای گردکردن اعشار (Rounding Errors) می‌شوند. برای سیستم‌های مالی و فروشگاهی، همیشه باید از `DecimalField` استفاده کرد.

• **مدل `ProductMedia` و فیلد `file` :**

چون قرار است هم عکس آپلود شود هم ویدیو، نمی‌توانستیم از `ImageField` استفاده کنیم، زیرا ایمیج فیلد ساختار ویدیو را بلاک می‌کند. بنابراین از `models.FileField` استفاده کردیم و آرگومان `validators=[validate_media_file]` را به آن پاس دادیم تا قوانین حجم و زمان روی آن اعمال شود.

• **متد `clean` در `ProductMedia` :**

این یک متد داخلی جنگو است که برای اعتبارسنجی‌های چند فیلدی یا منطقی استفاده می‌شود. ما با دستور `ProductMedia.objects.filter(product=self.product).count` بررسی می‌کنیم که این محصول در حال حاضر چند عکس یا ویدیوی فرعی دارد. اگر تعداد ۱۰ یا بیشتر بود، با پرتاب `ValidationError` جلوی ذخیره‌سازی را می‌گیریم. دستور `exclude(pk=self.pk)` برای این است که اگر ادمین خواست یک عکس از قبل آپلود شده را ویرایش کند، خودش را جزو آن ۱۰ تا حساب نکند و خطا ندهد.

• **متد `save` و دستور `self.full_clean()` :**

در جنگو، وقتی رکوردی از طریق پنل ادمین ثبت می‌شود، متد `clean` خودبه‌خود اجرا می‌شود. اما اگر بعداً کدی بنویسیم که از طریق API (مثلاً DRF) دیتا ذخیره کند، جنگو متد `clean` مدل را به صورت خودکار اجرا نمی‌کند! برای حل این گپ امنیتی، متد `save` را اورراید کردیم و داخلش `self.full_clean()` را صدا زدیم تا مطمئن شویم تحت هر شرایطی (چه ادمین، چه API) این ولیدیشن‌ها اجرا می‌شوند.



چرا از `OneToOneField` استفاده نکردیم؟

بیا ببینیم چرا این چالش را کاملاً کالبدشکافی کنیم تا تفاوت عمیق بین `OneToOneField` و `ForeignKey` را متوجه شویم و ببینیم چرا استفاده از `OneToOneField` در اینجا باعث خراب شدن سیستم گالری می‌شد.



تفاوت ساختاری در دیتابیس (SQL چیست؟)

وقتی ما در جنگو مدل می‌نویسیم، این مدل‌ها به جداول دیتابیس تبدیل می‌شوند. تفاوت این دو فیلد در سطح دیتابیس به این صورت است:

1. اگر از `OneToOneField` استفاده می‌کردیم (رابطه یک‌به‌یک):

در دیتابیس، جنگو ستون `product_id` را در جدول مدیا می‌سازد و روی آن یک قفل به نام `UNIQUE` (یکتا) می‌گذارد.

معنی یکتا بودن چیست؟

یعنی در کل جدول مدیا، فقط و فقط یک ردیف می‌تواند وجود داشته باشد که آی‌دی آن محصول (مثلاً محصول شماره ۵) در آن ثبت شده باشد.

نتیجه عملی:

شما برای لپ‌تاپ ایسوس (آی‌دی ۵) یک عکس فرعی آپلود می‌کنید. رکورد با موفقیت ذخیره می‌شود. حالا می‌خواهید عکس فرعی دوم یا یک ویدیو برای همان لپ‌تاپ آپلود کنید؛ دیتابیس MySQL فوراً خطا می‌دهد: `'Duplicate entry '5' for key 'product_id'` !

خلاصه:

با `OneToOneField` هر محصول فقط و فقط می‌توانست یک دانه فایل فرعی داشته باشد، نه ۱۰ تا!

2. چرا از `ForeignKey` استفاده کردیم (رابطه چندبه‌یک):

رابطه چندبه‌یک (Many-to-One) یعنی چندین ردیف در جدول مدیا می‌توانند به یک محصول اشاره کنند. در این حالت ستون `product_id` در جدول مدیا دیگر `UNIQUE` نیست.

به این جدول فرضی دیتابیس (جدول `ProductMedia`) نگاه کنید:

id	product_id (Foreign Key)	media_type	file_path
1	(لپ‌تاپ ایسوس) 5	image	gallery/pic1.jpg
2	(لپ‌تاپ ایسوس) 5	video	gallery/vid1.mp4
3	(لپ‌تاپ ایسوس) 5	image	gallery/pic2.jpg
4	(گوشی آیفون) 8	image	gallery/iphone.jpg

همانطور که می‌بینید، محصول شماره ۵ توانسته ۳ ردیف مختلف (دو عکس و یک ویدیو) در گالری داشته باشد. این دقیقاً همان چیزی است که ما می‌خواهیم؛

یعنی یک محصول بتواند یک گالری (مجموعه‌ای از فایل‌ها) داشته باشد.



پس چطور محدودیت "حداکثر ۱۰ فایل" را اعمال کردیم؟

اگر دیتابیس به لطف `ForeignKey` اجازه می‌دهد که بی‌نهایت عکس برای یک محصول ثبت شود، پس چطور جلوی ادمین را بگیریم که ۱۰۰ تا عکس آپلود نکند؟ اینجا کدهای پایتون و جنگو به کمک ما می‌آیند.

ما این محدودیت را در سطح **Business Logic (منطق کسب‌وکار)** اعمال کردیم، نه در سطح ساختار سخت‌گیرانه دیتابیس.

```
existing_media_count = ProductMedia.objects.filter(product=
self.product).count()
if existing_media_count >= 10:
    raise ValidationError("شما نمیتوانید بیشتر از ۱۰ فایل ف
رعی آپلود کنید")
```

کالبدشکافی منطق بالا:

هر بار که ادمین می‌خواهد عکس جدیدی آپلود کند، این کد ابتدا در دیتابیس جستجو می‌کند (`filter`) و می‌شمارد (`count`) که این محصول تا الان چقدر عکس دارد. اگر تعداد آن‌ها ۹ تا بود، اجازه می‌دهد عکس دهم ذخیره شود. اما اگر تعداد ۱۰ تا بود، پایتون ترمز را می‌کشد و با پرتاب `ValidationError` اجازه نمی‌دهد درخواست به دیتابیس برسد.



خلاصه و جمع‌بندی:

- برای چیزهایی که دقیقاً یکی هستند (مثل تصویر اصلی محصول)، فیلد را مستقیم در خود محصول می‌گذاریم (که عملاً حکم یک‌به‌یک دارد).
- برای چیزهایی که لیست یا گالری هستند (مثل تصاویر فرعی)، از `ForeignKey` استفاده می‌کنیم تا اجازه ثبت چند ردیف داده شود.
- برای اعمال محدودیت روی تعداد لیست، از متدهای معتبرسازی مثل `clean` در جنگو استفاده می‌کنیم.



حالا که این منطق قدرتمند را پیاده کردیم، باید تغییرات را به دیتابیس اعلام کنیم.

44- در ترمینال دستورات زیر را اجرا کنید

```
python manage.py makemigrations
python manage.py migrate
```

نتیجه شبیه زیر است:

```
$ python manage.py makemigrations products
Migrations for 'products':
  apps\products\migrations\0002_product_productmedia.py
    + Create model Product
    + Create model ProductMedia

$ python manage.py migrate
Operations to perform:
  Apply all migrations: accounts, admin, auth, contenttypes,
  customers, products, sessions
Running migrations:
  Applying products.0002_product_productmedia... OK
```



متصل کردن این ساختار به پنل ادمین

در حالت عادی، اگر بخواهیم برای یک محصول عکس فرعی آپلود کنیم، ادمین باید ابتدا به بخش **ProductMedia** برود، محصول را انتخاب کند، عکس را آپلود کند و ذخیره کند. این کار برای ۱۰ عکس یعنی ۱۰ بار رفت و آمد بین صفحات مختلف که اصلاً با معیارهای یک سیستم حرفه‌ای سازگار نیست.

جنگو برای این حل این مشکل قابلیت به نام **Inline Admin** دارد. با این قابلیت، گالری تصاویر دقیقاً در زیر صفحه ساخت خود محصول باز می‌شود تا ادمین همه چیز را در یک صفحه مدیریت کند.



پیاده‌سازی مدیریت یکپارچه محصول و گالری

45- فایل **apps/products/admin.py** را باز کنید و کدهای زیر را به انتهای آن اضافه کنید:

```

from django.contrib import admin
from .models import Category, Brand, ProductMedia

# کدهای قبلی مربوط به CategoryAdmin و BrandAdmin
# بدون تغییر اینجا بمانند...

# ۱. ساخت کلاس اینلاین برای گالری فرعی
class ProductMediaInline(admin.TabularInline):
    model = ProductMedia
    extra = 1
    # تعداد ردیفهای خالی که به صورت پیشفرض نمایش داده میشود
    # قفل کردن فرانتاند max_num = 10
    # همین روی حداکثر ۱۰ فایل فرعی

# ۲. ساخت کلاس مدیریت اصلی محصول
admin.register(Product)@
class ProductAdmin(admin.ModelAdmin):
    # الف) ستونهای نمایشی در جدول لیست محصول
    list_display = ['name', 'brand', 'category', 'price', 'inventory', 'created_at']

    # ب) باکس فیلتر در سمت راست پنل ادمین
    list_filter = ['category', 'brand', 'created_at']

    # ج) باکس جستجوی پیشرفته

```

```
search_fields = ['name', 'description']
```

د) پر شدن خودکار اسلاگ محصول بر اساس نام آن

```
prepopulated_fields = {'slug': ('name', slugify)}
```

هـ) تزریق گالری فرعی به انتهای صفحه محصول

```
inlines = [ProductMediaInline]
```



کالبدشکافی خط به خط کدهای ادمین (بدون Vibe Coding)

- `class ProductMediaInline(admin.TabularInline)` :
(پشته‌ای) `StackedInline` و (جدولی) `TabularInline` : ما در جنگو دو نوع اینلاین داریم. مدل جدولی فیلدها را در یک ردیف افقی و بسیار فشرده نشان می‌دهد که برای آپلود عکس و ویدیو عالی است و صفحه را بیش از حد طولانی نمی‌کند.
- `model = ProductMedia` :
`ProductMedia` به جنگو می‌گوییم این ردیف‌های درون برنامه‌ای، قرار است داده‌های جدول می‌فهمد که این `ForeignKey` را پر کنند. جنگو خودش به صورت هوشمند از روی رابطه مدیاها چطور به محصول متصل می‌شوند.
- `extra = 1` :
وقتی ادمین صفحه محصول را باز می‌کند، جنگو به طور پیش فرض ۱ ردیف خالی برای ردیف‌های Add another آپلود فایل جدید به او نشان می‌دهد. ادمین می‌تواند با دکمه بیشتری باز کند.
- `max_num = 10` :
این یک لایه امنیتی در فرانت‌اند پنل ادمین است. با این دستور، جنگو بعد از اینکه ادمین را مخفی می‌کند تا ادمین اصلاً نتواند Add another ۱۰ فایل فرعی را آپلود کرد، دکمه یادتان هست که در مرحله قبل، لایه امنیتی بک‌اند را هم (با) دکمه اضافه کردن را کلیک کند (در مدل قفل کرده بودیم؟ حالا سیستم از هر دو طرف کاملاً امن است `clean` متد).
- `list_filter = [...]` :
در پروژه‌های بزرگ با هزاران محصول، ادمین باید بتواند سریعاً محصولات را فیلتر کند.

این دستور یک سایدبار هوشمند در سمت راست ادمین می‌سازد تا محصولات بر اساس برند یا دسته‌بندی خاص جداسازی شوند.

- `search_fields = [...]`:
این فیلد یک باکس سرچ به بالای صفحه اضافه می‌کند. وقتی ادمین متنی را سرچ می‌کند، `description` و `name` روی فیلدهای `LIKE` با دستور SQL جنگو پشت صحنه یک کوئری می‌زند تا کالا را پیدا کند.



تست سناریوهای واقعی در پنل ادمین

حالا سرور پروژه را روشن کنید (`python manage.py runserver`) و به صفحه ادمین محصول بروید: `/http://127.0.0.1:8000/admin/products/product/add`

سه سناریوی زیر را برای چالش کشیدن کدهایتان تست کنید:

1. **تست اسلاگ و تصویر اصلی:** نام یک محصول را بنویسید (مثلاً: گوشی سامسونگ S24). ببینید چطور اسلاگ فارسی پر می‌شود. یک تصویر اصلی برای آن آپلود کنید.
2. **تست گالری فرعی:** در پایین صفحه، بخش گالری را می‌بینید. سعی کنید چند تصویر فرعی و یک ویدیوی کوتاه آپلود کنید و دکمه Save را بزنید. همه‌چیز باید بدون مشکل ذخیره شود.
3. **تست ولیدیتور (بسیار مهم):** یک ویدیوی طولانی (بالای ۲ دقیقه) یا یک فایل حجیم انتخاب کنید و سعی کنید محصول را ذخیره کنید. دیتابیس نباید رکورد را ثبت کند و جنگو باید دقیقاً بالای همان ردیف گالری، ارور فارسی که در متد `validate_media_file` نوشتیم را به ادمین نمایش دهد.



ارور `'FieldFile' object has no attribute 'temporary_file_path'`
....**AttributeError** at `/admin/products/product/1`

چرا این خطا (AttributeError) رخ داد؟

شما سعی کردید از متد `temporary_file_path()` استفاده کنید تا آدرس فایل ویدیو را به OpenCV بدهید. اما جنگو در مدیریت فایل‌های آپلودی رفتار دوگانه‌ای دارد:

1. **فایل‌های کوچک (زیر ۲.۵ مگابایت):** جنگو برای سرعت بیشتر، این فایل‌ها را اصلاً روی هارد دیسک ذخیره نمی‌کند! بلکه آن‌ها را مستقیماً در حافظه رم (RAM) با کلاسی به نام `InMemoryUploadedFile` نگه می‌دارد. چون فایل روی هارد نیست، طبیعتاً آدرس فیزیکی (Path) هم ندارد و متد `temporary_file_path` کار نمی‌کند.

2. فایل‌های بزرگ (بالای ۲.۵ مگابایت): جنگو این فایل‌ها را در پوشه موقت سیستم عامل با کلاسی به نام `TemporaryUploadedFile` ذخیره می‌کند. این فایل‌ها آدرس فیزیکی دارند.

3. فایل‌های از قبل ذخیره شده: اگر بخواهید محصولی را ویرایش کنید که از قبل ویدیویی دارد، جنگو یک شیء از نوع `FieldFile` به ولیدیتور می‌فرستد که آن هم متد `temporary_file_path` را ندارد.

از آنجا که کتابخانه (OpenCV) `cv2` فقط و فقط یک آدرس فایل فیزیکی روی هارد را می‌خواند و حافظه رم را نمی‌فهمد، سیستم کرش کرد.



راه حل معماری اصولی (استفاده از tempfile)

برای حل این مشکل بدون اینکه به تنظیمات پایه جنگو دست بزنیم، باید یک فایل موقت فیزیکی روی سرور بسازیم، فایل جنگو (چه در رم باشد چه روی هارد) را تکه‌تکه (Chunk) داخل آن بنویسیم، آدرس آن را به OpenCV بدهیم و در نهایت آن فایل موقت را حذف کنیم.

46- فایل `apps/products/models.py` را باز کنید و تابع `validate_media_file` را به شکل زیر اصلاح کنید:

```
import cv2
import os
import tempfile # این کتابخانه استاندارد پایتون باید اضافه شود
from django.db import models
from django.core.exceptions import ValidationError

def validate_media_file(file):
    # الف) بررسی حجم فایل (400 مگابایت به بالا)
    max_size_mb = 400
    max_size_bytes = max_size_mb * 1024 * 1024
    if file.size > max_size_bytes
```

```
raise ValidationError
    f"حجم فایل نم
    ی‌تواند بیشتر از {max_size_mb} مگا بایت باش
    د."
```

```
# (ب) بررسی مدت زمان ویدیو (اگر فایل ویدی
و بود)
file_name = file.name.lower
if file_name.endswith(('.mp4', '.mkv',
:'.avi', '.mov'))
```

```
# ۱. ساخت یک فایل موقت فیزیکی و امن
روی سیستم‌عامل
with tempfile.NamedTemporaryFile(delete=
:False, suffix='.mp4') as temp_video
# ۲. خواندن فایل جنگو به صورت تکه
ه تکه و نوشتن در فایل موقت
:()for chunk in file.chunks
temp_video.write(chunk)
temp_video_path = temp_video.name
```

```
:try
# ۳. دادن آدرس فایل فیزیکی موقت
به OpenCV
video = cv2.VideoCapture(temp_vi
deo_path)
```

```
frames = video.get(cv2.CAP_PROP_
FRAME_COUNT)
fps = video.get(cv2.CAP_PROP_FPS)
```



```

:if fps > 0
duration_seconds = frames /
fps
:if duration_seconds > 120
raise ValidationError
م"د
د.د
:finally
.۴ # پاکسازی (بسیار مهم): آزادسازی رم و حذف فایل موقت از روی هارد
:()if 'video' in locals
()video.release
if os.path.exists(temp_video_path):
h)
os.remove(temp_video_path)

```



کالبدشکافی کدهای جدید

- این ابزار درونی پایتون است. یک فایل فیزیکی کاملاً امن و `tempfile.NamedTemporaryFile`: گذاشتیم تا فایل فوراً بسته نشود `delete=False` یکتا روی هارد دیسک سرور می‌سازد. ما بدهیم OpenCV را به `(temp_video.name)` و بتوانیم آدرس آن
- ما کل فایل را یک‌جا در رم بارگذاری نمی‌کنیم (چون ممکن است ۴۰۰ `file.chunks()`: پایتون فایل را در بسته‌های کوچک (معمولاً، `chunks` مگابایت رم سرور را اشغال کند). با ۶۴ کیلوبایتی) می‌خواند و در فایل موقت می‌نویسد. این کار برای جلوگیری از پر شدن در پروداکشن الزامی است (Memory Overflow) حافظه
- **بلاک `try...finally`**: مهم‌ترین قانون در مدیریت فایل‌ها! اگر ویدیوی کاربر خراب باشد و OpenCV کرش کند، یا اگر خطای زمان بیش از ۲ دقیقه پرتاب شود (`raise` `ValidationError`)، اجرای کد متوقف می‌شود. بلاک `finally` تضمین می‌کند که **تحت هر شرایطی**، حتی اگر سرور خطا داد، فایل موقتی که ساختیم از روی هارد پاک شود تا هارد سرور به مرور زمان پر از ویدیوهای موقت نشود.

چرا الان زمان مناسبی برای Swagger نیست؟

- ۱. کمبود نقطه‌های پایانی (Endpoints): در حال حاضر، ما در کل پروژه فقط یک API واقعی داریم (همان `/GET/PATCH /api/customers/me` در فاز ۳). اگر الان ابزارهای OpenAPI (مثل `drf-spectacular` که استاندارد مدرن DRF است) را نصب کنیم، خروجی Swagger ما بسیار خالی و خسته‌کننده خواهد بود و نمی‌توانیم قدرت واقعی آن را ببینیم.
- ۲. از دست دادن تمرکز (Context Switching): ما الان دقیقاً در قلب **Product Domain** (فاز ۴) هستیم. مدل‌ها را معماری کردیم، ولیدیتورهای پیچیده نوشتیم و ادمین را یکپارچه کردیم. اگر الان وارد تنظیمات زیرساختی (Infrastructure) مثل Swagger بشویم، تمرکزمان از روی منطق کسب‌وکار به هم می‌خورد.
- ۳. غول مرحله آخر محصولات (مشکل N+1): ما هنوز Serializerها و Viewهای مربوط به محصولات را ننوشته‌ایم. در بخش نمایش محصولات، یک چالش وحشتناک به نام مشکل N+1 Query وجود دارد (وقتی یک کالا ۱۰ عکس دارد، دیتابیس برای هر عکس یک کوئری جداگانه می‌زند و سرور به شدت کند می‌شود). حل این مشکل اولویت بسیار بالاتری نسبت به مستندسازی دارد.



در نقطه‌ای هستیم که مفاهیم پیشرفته‌ی جنگو و DRF خودشان را نشان می‌دهند.

در این مرحله، هدف ما این است که دیتای محصول را به همراه برند، دسته‌بندی و گالری تصاویرش به یک JSON تمیز تبدیل کنیم. اما یک قانون مهم داریم: سریالایزر `Product` به سریالایزرهای دیگر وابسته است. یعنی قبل از اینکه بتوانیم محصول را JSON کنیم، باید به DRF یاد بدهیم که چطور یک "برند" یا "مدیا" را JSON کند.



قدم اول: ساخت Serializerهای پایه و `Product Serializer`

47- در مسیر `/apps/products` یک فایل جدید به نام `serializers.py` بسازید و کدهای زیر را با دقت وارد کنید:

```
from rest_framework import serializers
from .models import Category, Brand, Product, ProductMedia
```

```
# ۱. سریالایزر دسته بندی
class CategorySerializer(serializers.ModelSerializer):
    class Meta:
        model = Category
        fields = ['id', 'name', 'slug']
```

```
# ۲. سریالایزر برند
class BrandSerializer(serializers.ModelSerializer):
    class Meta:
        model = Brand
        fields = ['id', 'name', 'slug', 'image']
```

```
# ۳. سریالایزر گالری مدیا
class ProductMediaSerializer(serializers.ModelSerializer):
    class Meta:
        model = ProductMedia
        fields = ['id', 'media_type', 'file']
```

```
# ۴. سریالایزر اصلی محصول (Master Serializer)
class ProductSerializer(serializers.ModelSerializer):
    class Meta:
        model = Product
        fields = ['id', 'name', 'slug', 'category']
        # Nested Serializers (الف) برای فیلدهای ک
        # لید خارجی (ForeignKey)
        category = CategorySerializer(read_only=True)
```

```

brand = BrandSerializer(read_only=True)

# Nested Serializer برای رابطه معکوس (گالری)
media_gallery = ProductMediaSerializer(many=True, read_only=True)

class Meta
model = Product
] = fields
    , 'id'
    , 'name'
    , 'slug'
    , 'description'
    , 'price'
    , 'inventory'
    , 'main_image'
    , 'category'
    , 'brand'
    # اضافه کردن گالری به خروجی نهایی
    'media_gallery'
    'created_at'
[

```



کالبدشکافی خط به خط Serializer ها (بدون Vibe Coding)

- چرا ۳ سریالایزر اول را ساختیم؟
اگر در `ProductSerializer` فقط می‌نوشتیم `fields = ['category', 'brand']`، خروجی API فقط ID آن‌ها را برمی‌گرداند (مثلاً `"brand": 2`). اما در یک فروشگاه واقعی،

فرانت‌اند به نام و اسلاگ برند هم نیاز دارد تا آن را نمایش دهد. با پاس دادن `BrandSerializer` و `CategorySerializer` به متغیرهای `category` و `brand`، ما به DRF می‌گوییم: "به جای ID، کل آبجکت برند را به صورت تو در تو (Nested) برگردان".

- `media_gallery = ProductMediaSerializer(many=True, ...)` :
 - کلمه `media_gallery` : یادتان هست در فایل `models.py` وقتی `ProductMedia` را می‌ساختیم، نوشتیم `'related_name='media_gallery'` ؟ اینجا دقیقاً باید از همان نام استفاده کنیم تا DRF بفهمد باید برود و فایل‌های فرعی این محصول را پیدا کند.
 - پارامتر `many=True` : این حیاتی‌ترین کلمه در این خط است. چون گالری یک رابطه "چند به یک" است (یک محصول، ۱۰ عکس)، DRF باید بداند که با یک لیست (List) از داده‌ها طرف است، نه یک آبجکت تکی. اگر `many=True` را نگذارید، سیستم کرش می‌کند.

قدم دوم: غول مرحله آخر (مشکل N+1 در Views)

حالا که سریالایزر آماده است، باید View آن را بسازیم تا محصولات را به کاربر نمایش دهد (مثلاً در صفحه اصلی فروشگاه). اما اینجا یک تله بزرگ وجود دارد که برنامه‌نویسان تازه‌کار در آن می‌افتند: مشکل `N+1 Query`.

مشکل N+1 چیست؟

- فرض کنید ۱۰ محصول در دیتابیس داریم. اگر در View فقط بنویسیم `Product.objects.all()`، جنگو ۱ کوئری می‌زند تا ۱۰ محصول را بیاورد. سپس سریالایزر برای محصول اول می‌گوید: "من عکس‌های گالری‌ات را می‌خواهم!" -> جنگو ۱ کوئری دیگر می‌زند. محصول دوم: "عکس‌های من؟" -> ۱ کوئری دیگر. برای ۱۰ محصول، دیتابیس ۱۱ بار $(1 + 10)$ درگیر می‌شود. حالا فرض کنید دیجی‌کالا در یک صفحه ۱۰۰ محصول با برند و دسته‌بندی و گالری لود کند... سرور با ۳۰۱ کوئری نابود می‌شود!
- راه حل معماری دیتابیس در جنگو:

- استفاده می‌شود. جنگو در (مثل برند و دسته) `ForeignKey` برای روابط: `select_related` می‌زند و همه را در همان ۱ کوئری اول می‌آورد `SQL JOIN` سطح دیتابیس یک
- برای روابط معکوس و لیست‌ها (مثل گالری تصاویر) استفاده: `prefetch_related` (SQL در `IN` با دستور) می‌شود. جنگو تمام گالری‌های مرتبط را فقط با ۱ کوئری اضافه می‌آورد و در رم پایتون آن‌ها را به هم می‌چسباند.

48- در مسیر `/apps/products` فایل `views.py` را باز کنید (یا بسازید) و کدهای بهینه‌شده زیر را بنویسید:

```

from rest_framework.generics import ListAPIV
iew
from rest_framework.permissions import Allow
Any
from .models import Product
from .serializers import ProductSerializer

:class ProductListView(ListAPIView)
"""
    API دریافت لیست تمام محصولات فروشگاه
    آزاد برای تمام کاربران (بدون نیاز به لاگین)
    """
    # permission_classes = [AllowAny]
    # می‌توانند محصولات را ببینند
    serializer_class = ProductSerializer

    # QuerySet کاملاً بهینه‌سازی شده برای جلوگیری
    # از مشکل N+1
    queryset = Product.objects.select_related(
        'category',
        'brand'
    ).prefetch_related(
        'media_gallery'
    ).all()

```

کالبدشکافی View

- بنویسیم و `get` به جای اینکه خودمان به صورت دستی مثل فاز ۳ متد `ListAPIView` استفاده کردیم. این کلاس به DRF های آماده Generic View برگردانیم، از `Response` صورت خودکار کوئری ست را می‌گیرد، آن را به سرپالایزر می‌دهد و در صورت تنظیم خروجی را صفحه‌بندی می‌کند، (که در تنظیمات پایه انجام دادیم) Pagination بودن
- کاتالوگ محصولات باید برای `(IsAuthenticated)` برعکس پروفایل مشتری: `AllowAny` عموم مردم و موتورهای جستجو (گوگل) باز باشد تا سئو سایت کار کند
- **بهینه‌سازی Query:** با ترکیب `select_related` و `prefetch_related`، ما تعداد کوئری‌های دیتابیس را برای لود کردن ۱۰۰ محصول از ۳۰۱ کوئری به فقط ۲ کوئری کاهش دادیم! این یعنی کد شما در سطح یک برنامه نویس سنیور و آماده برای ترافیک پروداکشن است.



قدم نهایی: اتصال به URL

49- در مسیر `/apps/products` یک فایل به نام `urls.py` بسازید و مسیر زیر را تعریف کنید:

```
from django.urls import path
from .views import ProductListView

urlpatterns = [
    path('', ProductListView.as_view(), name='product-list')
]
```

50- این URL را به روت اصلی API در فایل `apps/api/urls.py` متصل کنید:

فایل `apps/api/urls.py` را باز کنید و مسیر `/products` را به آن اضافه کنید:

```
from django.urls import path
from .views import ProductListView
```

```
] = urlpatterns
path('', ProductListView.as_view(), name
,='product-list')
[
```

حالا می‌توانید در Postman (بدون نیاز به توکن)، به آدرس **GET** [/http://127.0.0.1:8000/api/products](http://127.0.0.1:8000/api/products) درخواست بزنید و محصولاتی که در پنل ادمین ساخته‌اید را با یک JSON بسیار زیبا، کامل (شامل برند و گالری) و دیتابیزی کاملاً بهینه‌شده مشاهده کنید.

ویژگی	select_related	prefetch_related
نوع رابطه مناسب	ForeignKey, OneToOne	ManyToMany, Reverse ForeignKey
کارکرد در دیتابیس	از طریق SQL JOIN	از طریق چند کوئری مجزا و IN فیلتر با
تعداد کوئری‌ها	همیشه ۱ کوئری	بیش از ۱ کوئری (معمولاً ۲ کوئری)
محل پردازش اتصال	(SQL) داخل خود دیتابیس	(Django ORM) داخل حافظه پایتون

51- داخل ترمینال این رو بنویس تا پکیج **django debug toolbar** نصب بشه

```
pipenv install django-debug-toolbar
```

51- داخل ترمینال این رو بنویس تا پکیج **django debug toolbar** نصب بشه

```
pipenv install django-debug-toolbar
```


52- داخل فایل `base.py` چک کن باشه

```
Check for Prerequisites #
] = INSTALLED_APPS
... #
, "django.contrib.staticfiles"
... #
[
"/STATIC_URL = "static
```

53- داخل فایل `base.py` اضافه کن

```
] = INTERNAL_IPS
... #
, "127.0.0.1"
... #
[

] = INSTALLED_APPS
... #
, "debug_toolbar"
... #
[

] = MIDDLEWARE
... #
debug_toolbar.middleware.DebugToolbarMi"
, "ddleware
```

```
... #  
[
```

54- داخل فایل config/urls.py اضافه کن

```
from django.urls import include, path  
from debug_toolbar.toolbar import debug_toolbar_urls  
  
urlpatterns =  
    the rest of your URLconf goes here ... #  
    ...  
    debug_toolbar_urls + [
```

55- داخل فایل requirements.txt اضافه کن و داخل گیت هاب push کن

```
pipenv requirements > requirements.txt
```

56- برای تست کردن اپلیکیشن debug toolbar رو disable کن
Disable the toolbar when running tests (optional)

```
TESTING = "test" in sys.argv or "PYTEST_VERSION" in os.environ
```

```

:if not TESTING
] = INSTALLED_APPS
,INSTALLED_APPS*
,"debug_toolbar"
[
] = MIDDLEWARE
debug_toolbar.middleware.DebugToolb"
,"arMiddleware
,MIDDLEWARE*
[

```

You should also modify your URLconf file:

```

from django.conf import settings

:if not settings.TESTING
from debug_toolbar.toolbar import debug_
toolbar_urls

] = urlpatterns
,urlpatterns*
()debug_toolbar_urls + [

```

پایان Part-4